



**POLYTECHNIQUE
MONTREAL**

LOG8100 – DevSecOps - Opérations et dev. logiciel sécur

Automne 2024

TP 2

Groupe 1

1976567 – Chenak Alaa eddine

2149119 – Mahfoud Brizini

2122907 Christine Audrey Njall

2275885 – Arien Andalibi-Abadan

Soumis à : Antoine Boucher

12 septembre 2024

Lien du repo (public)	3
Introduction	3
Analyse de Sécurité	3
OWASP ZAP et Orchestron	3
Introduction	3
Identification des alertes	5
1. Absence de Jetons Anti-CSRF	5
2. CSP: Wildcard Directive	5
3. Content Security Policy (CSP) Header Not Set	6
4. Missing Anti-clickjacking Header	6
Description du pipeline d'intégration continue (CI)	7
Mise en place du Pipeline CI pour Automatiser les Tests de Sécurité	7
A. Pipeline CI pour OWASP ZAP avec Orchestron	7
B. Pipeline CI pour SonarQube	10
C. Automatisation de la Gestion des Dépendances avec Dependabot:	12
Configuration pour la Publication Automatique d'Images Docker sur Docker Hub	14
Description du pipeline de déploiement continu (CD)	16
1.Étapes pour déployer PostgreSQL dans Azure Container	17
2.Étapes pour déployer DVNA dans Azure Container	20
Conclusion	23

Lien du repo (public)

<https://github.com/juceTP/dvna>

<https://jucetp.github.io/dvna/>

Introduction

L'objectif de ce projet est de réaliser une analyse de la sécurité d'une application Web vulnérable à l'aide d'OWASP ZAP, en identifiant et documentant les vulnérabilités critiques. Cette analyse sera suivie de la mise en place d'un pipeline CI/CD automatisé utilisant des outils tels que GitHub Actions ou Jenkins pour intégrer les tests de sécurité et gérer les dépendances, avec un déploiement continu sur Docker Hub et GitHub Pages.

Analyse de Sécurité

OWASP ZAP et Orchestron

Introduction

Afin de réaliser les tests de pénétrations et ensuite procéder à l'analyse de la sécurité de l'application, nous avons lancé le site web dans un conteneur à l'aide de la commande suivante:

```
docker run --name dvna -p 9090:9090 -d appsecco/dvna:sqlite
```

Ensuite, nous avons une attaque automatique, sur l'adresse suivante:
<http://localhost:9090/login>.

Pour ce qui est de Orchestron, nous avons généré un rapport en XML à partir de OWASP ZAP après avoir créé notre conteneur Orchestron afin d'avoir accès au logiciel localement. Ensuite, nous avons lancé la commande suivante:

```
curl -H "Secret-Key: 1fdca930093509040011e76c4f9c0cbbaac062d0" -H  
"Access-Key: a183e43c9290f8e51e0ab17ecdb633f92d957907" -v -F  
file=@<file_path>  
http://localhost/api/webhooks/post/bebb687b-0d01-43fd-ae45-db53218e05db/
```

cette commande envoie un fichier à un webhook local en utilisant deux clés d'authentification pour l'accès et la sécurité, tout en affichant les détails de la transaction.

Finalement, nous avons généré deux rapports résumant les vulnérabilités trouvées et les risques associés.

Résultats Automatique

		Confidence				
Risk		User Confirmed	Haut	Moyen	Faible	Total
	Haut	0 (0,0 %)	0 (0,0 %)	0 (0,0 %)	0 (0,0 %)	0 (0,0 %)
	Moyen	0 (0,0 %)	2 (15,4 %)	2 (15,4 %)	1 (7,7 %)	5 (38,5 %)
	Faible	0 (0,0 %)	0 (0,0 %)	4 (30,8 %)	0 (0,0 %)	4 (30,8 %)
	Pour information	0 (0,0 %)	2 (15,4 %)	0 (0,0 %)	2 (15,4 %)	4 (30,8 %)
	Total	0 (0,0 %)	4 (30,8 %)	6 (46,2 %)	3 (23,1 %)	13 (100%)

Figure 1: Résultats du test d'attaque sur l'application avec OWASP ZAP

L'attaque du site web permet d'identifier 13 alertes au total. On comptabilise 5 de risque *moyen* dont le niveau de confiance est de élevé, moyen et faible pour respectivement 2, 2, 1 alerte. De plus, on identifie 4 alertes de risque faible dont le niveau de confiance est moyen. Le reste sont de risque informationnel.

Résultats Orchestron

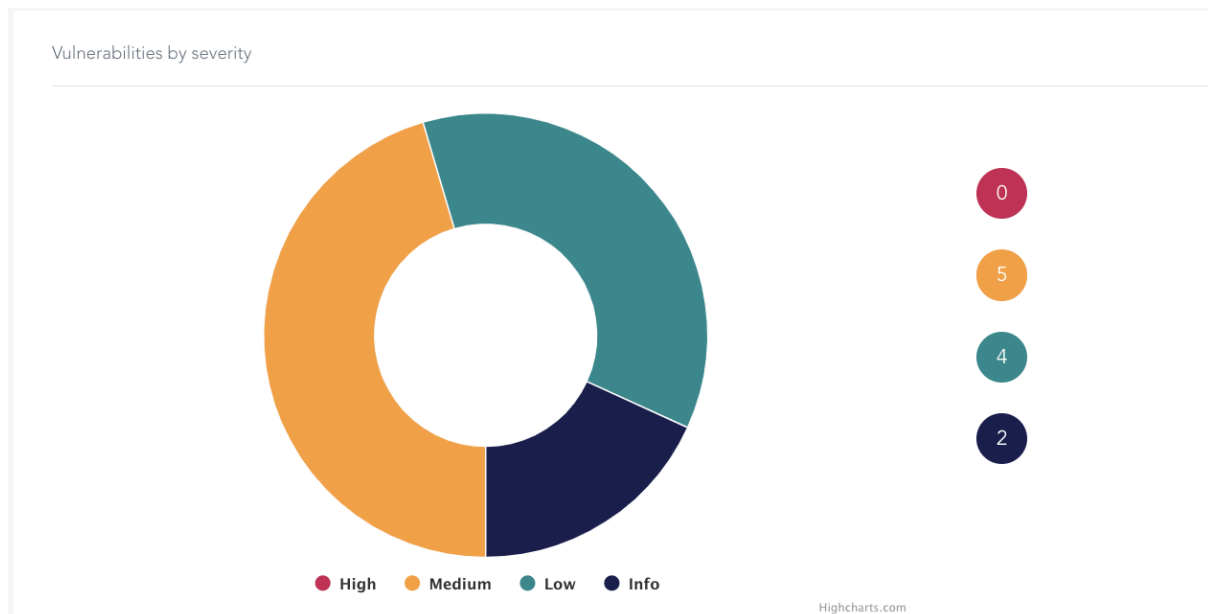


Figure 2: Résultats du test d'attaque sur l'application Orchestron
Sur Orchestron, on indique 5 vulnérabilités de gravité medium, 4 vulnérabilités de gravité faible et 2 de gravité informationnelle.

Identification des alertes

Les problèmes listés dans cette section relèvent principalement du domaine de la **Security Misconfiguration (A05:2021)** dans l'OWASP Top Ten.

1. Absence de Jetons Anti-CSRF

a. Nature de la vulnérabilité : *Cross-Site Request Forgery (CSRF)*

b. Localisation : *forgotpw.ejs*

```
<form          id="resetPasswordStart"          name="resetPasswordStart"
action="/forgotpw" method="post">
```

c. Gravité : Moyenne

d. Risques Associés :

- Un attaquant peut forcer un utilisateur authentifié à effectuer des actions malveillantes à son insu.
- Peut entraîner des modifications non autorisées de données (modification de paramètres de compte, transactions financières, etc.).
- Compromet l'intégrité des sessions utilisateurs.
- Risque élevé de pertes financières, de fuite de données, ou de prise de contrôle de comptes.

2. CSP: Wildcard Directive

a. Nature de la vulnérabilité : *Cross-Site Scripting (XSS) / Relaxed Security Policy*

b. Localisation : *sitemap.xml*
default-src 'self'

c. Gravité : Moyenne

d. Risques Associés:

- Permet à un attaquant d'injecter des scripts malveillants (XSS), mettant en péril la confidentialité des utilisateurs.
- Expose l'application à des contenus non sécurisés provenant de sources non fiables.
- Augmente les risques de vol de données sensibles (cookies, tokens d'authentification).
- Peut entraîner le défigurement du site ou la distribution de logiciels malveillants.

3. Content Security Policy (CSP) Header Not Set

a. Nature de la vulnérabilité : *Cross-Site Scripting (XSS) / Relaxed Security Policy*

b. Localisation: *login.ejs*

c. Gravité : Moyenne

d. Risques Associés:

- L'absence d'une politique de sécurité (CSP) ouvre la porte aux attaques de type XSS (Cross-Site Scripting).
- Risque de vol de sessions utilisateurs via l'exécution de scripts malveillants.
- Permet le chargement de ressources non sécurisées, augmentant le risque de distribution de contenu malveillant.
- Expose l'application à des attaques par injection de code provenant de domaines non approuvés.

4. Missing Anti-clickjacking Header

a. Nature de la vulnérabilité : *Clickjacking*

b. Localisation: *forgotpw.ejs*

c. Gravité: Moyenne

d. Risques Associés:

- Un attaquant peut tromper un utilisateur pour qu'il clique sur des éléments invisibles, causant des actions involontaires.
- Risque de détournement de sessions utilisateur pour des actions malveillantes (comme la validation d'une transaction non autorisée).
- Peut compromettre la sécurité des utilisateurs en les forçant à effectuer des actions dangereuses ou non désirées.
- Affecte l'intégrité et la confiance des utilisateurs dans l'application.

Description du pipeline d'intégration continue (CI)

1. Ajout et Configuration des Secrets GitHub

Avant de commencer la configuration du workflow CI, nous avons ajouté des secrets pour sécuriser les informations sensibles, telles que les clés API, les jetons d'accès, et les identifiants d'utilisateur.

1. Accéder au dépôt GitHub.
2. Aller dans l'onglet Settings.
3. Sélectionner Secrets and Variables > Actions.
4. Ajouter les secrets nécessaires en cliquant sur New repository secret.
 - DOCKER_USERNAME : Nom d'utilisateur pour Docker Hub.
 - DOCKER_PASSWORD : Mot de passe pour Docker Hub.
 - SONAR_TOKEN : Jeton d'accès pour SonarQube.

Mise en place du Pipeline CI pour Automatiser les Tests de Sécurité

A. Pipeline CI pour OWASP ZAP avec Orchestron

Le pipeline CI est conçu pour exécuter des tests de sécurité automatisés à chaque push de code vers la branche principale du projet. OWASP ZAP est utilisé pour effectuer un scan de sécurité sur l'application web, et Orchestron est employé pour gérer et orchestrer ces tests. En cas de détection d'une vulnérabilité, une issue GitHub est automatiquement créée.

Configuration dans GitHub Actions :

1. Créer un fichier **.github/workflows/zap_scan.yml** avec le contenu suivant:

```
1  name: "ZAP Baseline Scan"
2
3  on: [push]
4
5  jobs:
6    zap_scan:
7      runs-on: ubuntu-latest
8      name: Scan the web application
9      permissions:
10       issues: write
11      steps:
12       - name: Checkout
13         uses: actions/checkout@v4
14         with:
15           ref: master
16
17       - name: Run ZAP Scan
18         id: zap_scan
19         uses: zapproxy/action-baseline@v0.13.0
20         with:
21           token: ${ secrets.GITHUB_TOKEN }
22           docker_name: 'ghcr.io/zaproxy/zaproxy:stable'
23           target: 'https://dvna.blueisland-aa8261f7.canadaeast.azurecontainerapps.io'
24           cmd_options: '-a'
```

Figure 3: Configuration du pipeline GitHub Actions pour le scan ZAP

2. Processus du Pipeline CI ZAP avec Orchestron

Voici les différentes étapes qui constituent le pipeline CI mis en place pour automatiser les tests de sécurité :

1. Push vers la branche master : Lorsqu'un développeur soumet du code (push) vers la branche principale, GitHub Actions est déclenché.
2. Déclenchement de GitHub Action : Le workflow de GitHub Actions est automatiquement lancé.
3. Exécution du Scan ZAP : Le scan ZAP est exécuté en utilisant un conteneur Docker pour analyser la sécurité de l'application web.
4. Orchestron : Orchestron gère l'ensemble du processus de scan et collecte les résultats.
5. Archivage des résultats : Les résultats du scan sont archivés dans le pipeline GitHub Actions.

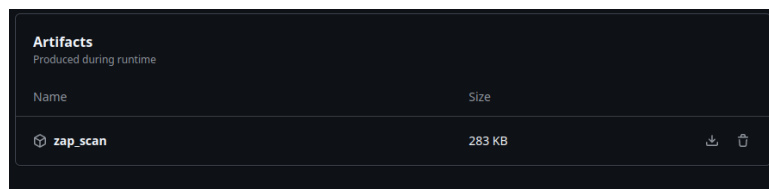


Figure 4: Résultat du scan ZAP dans l'artifactory

6. Création d'une issue GitHub : Si des vulnérabilités sont détectées, une issue est automatiquement créée dans GitHub, permettant aux développeurs de traiter le problème.

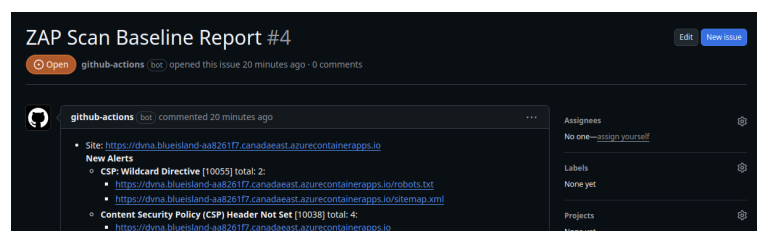


Figure 5: Issue automatisée créée par le code de scan ZAP

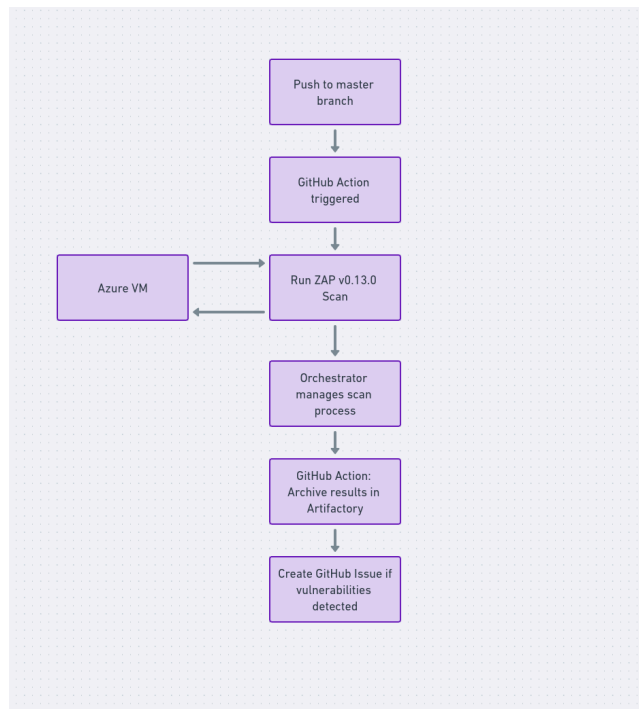


Figure 6: Diagramme d'État du Zap scan

B. Pipeline CI pour SonarQube

Le pipeline CI pour SonarQube est conçu pour analyser la qualité du code source en s'assurant qu'il respecte les normes de qualité de code telles que la détection des bugs, des vulnérabilités de sécurité, et du code dupliqué. SonarQube fournit un rapport détaillé après chaque soumission de code, permettant aux développeurs d'améliorer la qualité continue de l'application.

SonarQube est intégré à GitHub Actions pour déclencher automatiquement l'analyse lors de chaque push request sur la branche principale.

1. Créer un fichier `.github/workflows/sonarqube-scan.yml` avec le contenu suivant :

```
1  name: Build
2  on:
3    push:
4      branches:
5        - master
6    pull_request:
7      types: [opened, synchronize, reopened]
8  jobs:
9    sonarcloud:
10     name: SonarCloud
11     runs-on: ubuntu-latest
12     steps:
13       - uses: actions/checkout@v4
14         with:
15           fetch-depth: 0
16       - name: SonarCloud Scan
17         uses: SonarSource/sonarcloud-github-action@master
18         env:
19           GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
20           SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
```

Figure 7: Configuration du pipeline GitHub Actions pour l'analyse SonarCloud

Pour assurer la sécurité et l'authentification avec SonarQube et GitHub, les secrets suivants ont été ajoutés à GitHub :

SONAR_TOKEN : Jeton d'accès pour SonarQube permettant d'exécuter les analyses de qualité.

2.Processus du Pipeline CI SonarQube

Description du Workflow :

Voici les différentes étapes qui constituent le pipeline CI pour SonarQube :

1. Push : Le pipeline est déclenché lorsque du code est poussé sur la branche `master` ou lorsqu'une pull request est ouverte ou synchronisée.
2. Checkout du code : Le code est extrait de la branche cible (master) via l'action `checkout@v4`.
3. Scan SonarCloud : SonarQube analyse la qualité du code source et produit un rapport complet sur les éventuelles failles ou zones à améliorer.
4. Résultats : Les résultats de SonarQube sont disponibles dans l'interface GitHub, ce qui permet aux développeurs de consulter les métriques de qualité et d'apporter des

améliorations si nécessaire.

give issue write permission to zap
master f498c3b

> Publish Docker Image
on: push 1

> CodeQL Scan
on: push 2

> Build
on: push

> ZAP Baseline Scan
on: push 1

✓ SonarCloud

✓ SonarCloud Code Analysis

SonarCloud / SonarCloud Code Analysis
succeeded 41 minutes ago in 31s

Quality Gate passed

Issues
✓ 64 New issues
🛡️ 0 Accepted issues

Measures
✓ 8 Security Hotspots
✓ 0.0% Coverage on New Code
✓ 0.0% Duplication on New Code

[See analysis details on SonarCloud](#)

[View more details on SonarCloud](#)

Figure 8: Résultats de l'analyse de code **SonarQube**

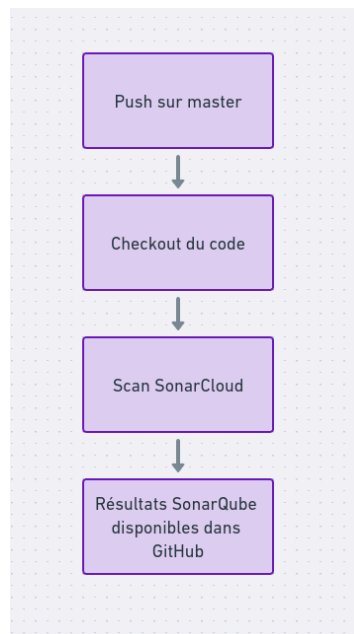


Figure 9: Diagramme d'État de **SonarQube**

C. Automatisation de la Gestion des Dépendances avec Dependabot:

Dependabot est un outil intégré à GitHub qui permet de gérer automatiquement . Ce mécanisme assure que les dépendances sont régulièrement mises à jour afin de réduire les vulnérabilités potentielles et d'assurer la compatibilité avec les versions les plus récentes.

1. Créer un fichier `.github/dependabot.yml` avec le contenu suivant :

```
1 # To get started with Dependabot version updates, you'll need to specify which
2 # package ecosystems to update and where the package manifests are located.
3 # Please see the documentation for all configuration options:
4 # https://docs.github.com/code-security/dependabot/dependabot-version-updates/configuration-options-for-the-dependabot.yml-file
5
6 version: 2
7 updates:
8   - package-ecosystem: "npm" # See documentation for possible values
9     directory: "/" # Location of package manifests
10     schedule:
11       interval: "weekly"
12
```

Figure 10: Configuration des paramètres de sécurité du projet GitHub avec Dependabot

Dependabot analysera automatiquement les dépendances du projet chaque semaine et créera des pull requests pour les mises à jour nécessaires.

Activation des Paramètres :

1. *Dependabot Alerts* : Activez cette option pour permettre à GitHub de détecter automatiquement les vulnérabilités dans vos dépendances.
2. *Dependabot security updates* : Autorisez les mises à jour automatiques de sécurité.
3. *Dependabot version updates* : Activez les mises à jour de version régulières pour maintenir les dépendances à jour.

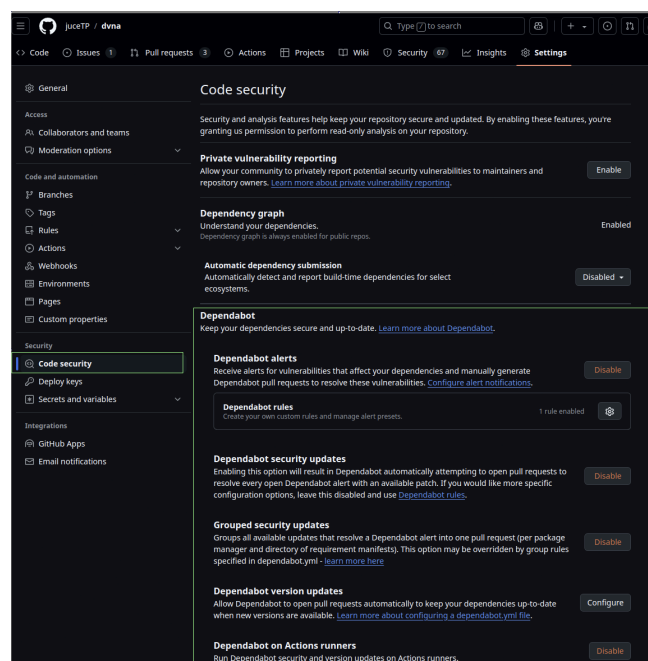


Figure 11: Interface de gestion des dépendances avec Dependabot dans GitHub

2. Processus du Pipeline CI ZAP avec Orchestron:

1. Dependabot se lance : Dependabot démarre son processus dans le dépôt de code.
2. Vérifie les dépendances chaque semaine : Dependabot effectue un contrôle hebdomadaire des dépendances du projet pour voir si elles sont à jour.
3. Identifie les dépendances obsolètes et les vulnérabilités : Il détecte les dépendances qui ne sont plus à jour ou qui présentent des risques de sécurité.
4. Crée une pull request pour les mises à jour : Dependabot propose automatiquement une mise à jour en créant une pull request (PR) avec les nouvelles versions des dépendances.
5. Génère un rapport sur GitHub : Un rapport détaillé est généré sur GitHub, récapitulant les mises à jour et les vulnérabilités corrigées.

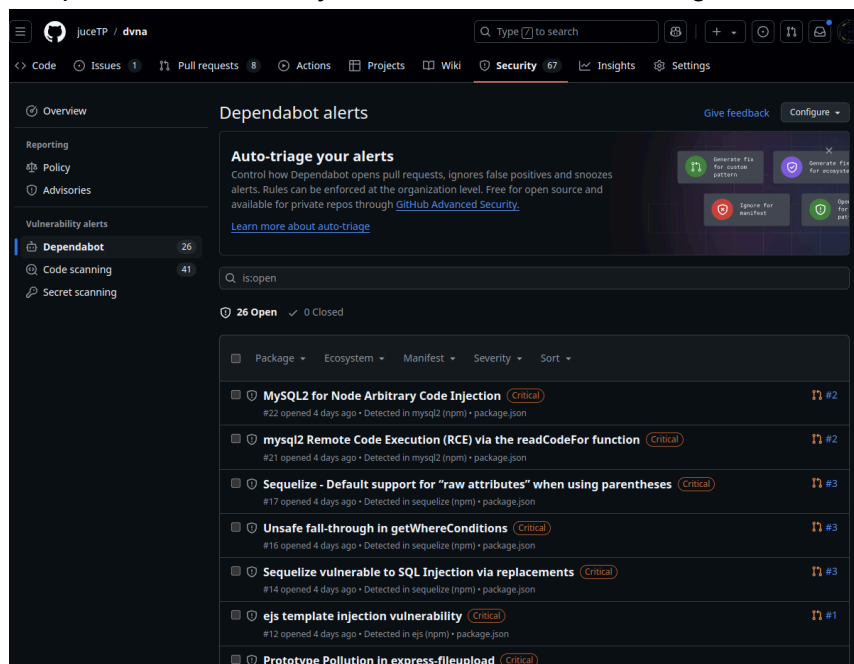


Figure 12: Vue des alertes Dependabot sur GitHub affichant les vulnérabilités et les mises à jour des dépendances.

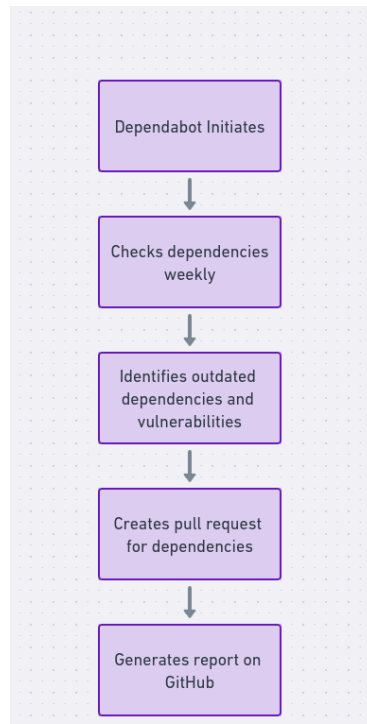


Figure 13 : Diagramme d'état du processus de gestion des dépendances par Dependabot.

Configuration pour la Publication Automatique d'Images Docker sur Docker Hub

Le pipeline CI présenté ici utilise GitHub Actions pour créer automatiquement une image Docker à chaque **push** vers la branche principale du projet, puis publie cette image sur Docker Hub.

1. Créer un fichier `.github/workflows/docker-build-push.yml` avec le contenu suivant :

```

1 name: Publish Docker Image
2
3 on:
4   push:
5     branches:
6       - master
7
8 jobs:
9   build:
10    runs-on: ubuntu-latest
11
12    steps:
13      # Checkout the repository
14      - name: Checkout code
15        uses: actions/checkout@v3
16
17      # Install Docker Compose
18      - name: Install Docker Compose
19        run: |
20          sudo curl -L "https://github.com/docker/compose/releases/download/v2.19.2/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
21          sudo chmod +x /usr/local/bin/docker-compose
22          docker-compose --version
23
24      # Log in to Docker Hub
25      - name: Log in to Docker Hub
26        uses: docker/login-action@v2
27        with:
28          username: ${{ secrets.DOCKER_USERNAME }}
29          password: ${{ secrets.DOCKER_PASSWORD }}
30
31      # Build and push Docker images using docker-compose
32      - name: Build and push Docker images
33        run: |
34          docker-compose build
35          docker-compose push
36
37      # Optional: Logout of Docker Hub
38      - name: Log out from Docker Hub
39        run: docker logout
  
```

Figure 14 : Pipeline CI pour la création et la publication d'images Docker sur Docker Hub

Secrets à configurer dans GitHub Actions :

- `DOCKER_USERNAME` : Nom d'utilisateur Docker Hub.
- `DOCKER_PASSWORD` : Mot de passe Docker Hub.

2. Processus Automatisé de Construction et Déploiement d'Images Docker avec Docker Compose dans un Pipeline CI

1. Push du code dans master : Lorsqu'un développeur soumet du code (push) vers la branche principale, GitHub Actions est déclenché.
2. Installer Docker Compose: Docker Compose est installé via un script `curl`, et les permissions d'exécution sont définies pour Docker Compose.
3. Connexion à Docker Hub : Utilise l'action `docker/login-action@v2` pour se connecter à Docker Hub avec les informations d'identification fournies dans les secrets GitHub :
 - a. `DOCKER_USERNAME` : Nom d'utilisateur Docker Hub.
 - b. `DOCKER_PASSWORD` : Mot de passe Docker Hub.
4. Création et publication des images Docker : Cette étape utilise Docker Compose pour construire l'image à partir du fichier `docker-compose.yml`, puis la publie sur Docker Hub en utilisant la commande `docker-compose push`.
5. Déconnexion de Docker Hub : Pour sécuriser davantage la session, cette étape se déconnecte de Docker Hub après la publication.

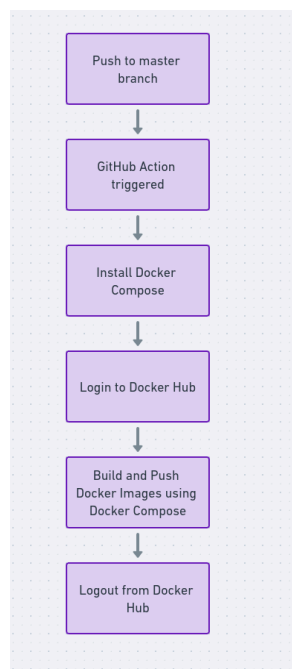


Figure 15 : Diagramme d'état Processus de déploiement avec Docker Compose et Docker Hub via GitHub Actions

Description du pipeline de déploiement continu (CD)

Le projet consiste à déployer deux applications conteneurisées dans des **Azure Container Instances** (ACI). La première instance héberge l'application **DVNA (Damn Vulnerable Node Application)**, une application web intentionnellement vulnérable pour tester la sécurité. La deuxième instance héberge une base de données **PostgreSQL**, qui sera utilisée par l'application DVNA pour stocker et gérer les données. L'objectif est de mettre en place un déploiement continu qui déploie la dernière version des conteneurs DVNA et PostgreSQL à partir de DockerHub dans un environnement de production. Ce pipeline doit permettre d'importer et de déployer automatiquement les conteneurs les plus récents dès qu'ils sont disponibles, sans nécessiter d'intervention manuelle.

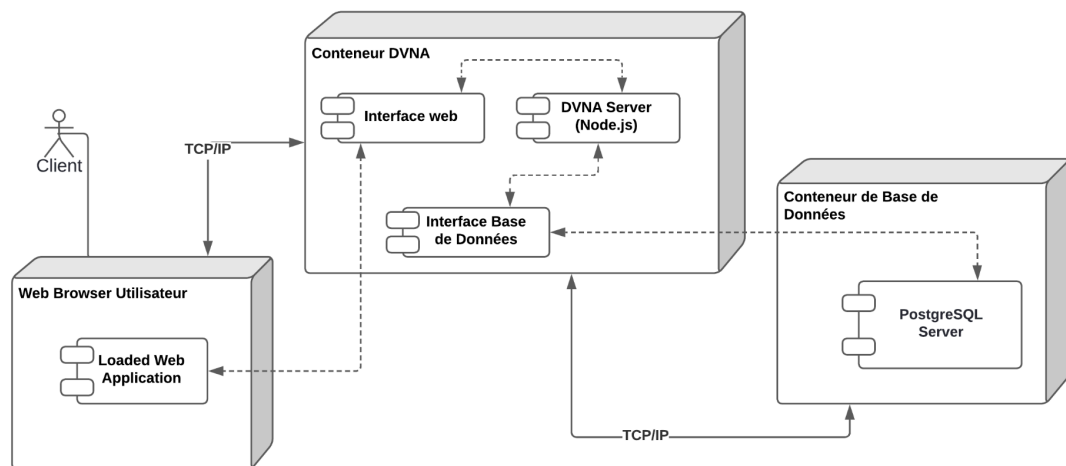


Figure 16: Diagramme de déploiement DVNA

Pour réaliser cela, nous avons d'abord déployé PostgreSQL avec **Azure Container Instances** (ACI). Voici les étapes principales pour ce déploiement :

1.Étapes pour déployer PostgreSQL dans Azure Container

1. **Nommer le conteneur** : Dans le champ **Name**, on a saisi **postgres** pour identifier clairement votre instance PostgreSQL dans Azure.
2. **Choisir la source de l'image Docker** : On a sélectionné **Docker Hub** comme source d'images. Cela signifie qu'on va récupérer une image PostgreSQL depuis le registre Docker public.
3. **Sélectionner le type d'image** : On a opté pour une image **Public**. Cela signifie que l'image PostgreSQL qu'on utilise est accessible à tous sur Docker Hub, sans restrictions d'accès.
4. **Spécifier l'image et la version de PostgreSQL** : On a indiqué **postgres:13**, ce qui signifie qu'on va déployer la version 13 de PostgreSQL à partir de l'image disponible sur Docker Hub.
5. **Configurer les variables d'environnement pour PostgreSQL** : On a ajouté trois variables d'environnement importantes :
 - **POSTGRES_DB** : Le nom de la base de données que PostgreSQL doit créer au démarrage est **mydatabase**.
 - **POSTGRES_USER** : L'utilisateur principal de la base de données est **chenak**.
 - **POSTGRES_PASSWORD** : Un mot de passe est défini pour cet utilisateur assurant la sécurité de l'accès

Create Container App

Use quickstart image ☐

Container details

Name *

Image source ☐ Azure Container Registry ☒ Docker Hub or other registries

Image type ☒ Public ☐ Private

Registry login server *

Image and tag *

Command override

Arguments override

Development stack-specific features

When you select a specific development stack, you get additional features tailored to that stack—optimizing Container Apps to perform for your unique settings.

Development stack

Container resource allocation

Choose the workload profile for this app. You can adjust the CPU and memory allocation for this app up to the workload profile limit. [Learn more](#)

Workload profile *

CPU and Memory *

Environment variables

Name	Value	Delete
POSTGRES_DB	mydatabase	
POSTGRES_USER	chenak	
POSTGRES_PASSWORD	*****	
Enter name	Enter value	

Figure 17: Configuration d'une instance PostgreSQL dans Azure Container App

Activer Ingress :

1. **Enabled**: Permettre au conteneur PostgreSQL de recevoir des connexions réseau.
2. **Limiter le trafic** : Sélectionner "Limited to Container Apps Environment" pour restreindre l'accès uniquement aux applications dans le même environnement Azure.
3. **Sélectionner TCP** : Choisir TCP comme type de protocole réseau, car PostgreSQL utilise ce protocole.
4. **Configurer le port cible** : Définir le port **5432** comme port d'écoute de PostgreSQL (port par défaut pour PostgreSQL).
5. **Configurer le port exposé** : Exposer également le port **5432** pour permettre aux services internes de se connecter à PostgreSQL.

Home > Container Apps >

Create Container App

Basics Container **Ingress** Tags Review + create

Application ingress settings

Enable ingress for applications that need an HTTP or TCP endpoint.

Ingress ⓘ ☒ Enabled

Ingress traffic

☒ Limited to Container Apps Environment
Select this option if you want to restrict traffic to this container app from within the Container App Environment

☐ Accepting traffic from anywhere
Select this option if you want to allow traffic to this container app from anywhere

Ingress type ⓘ

☐ HTTP

☒ TCP

Target port ⓘ

Exposed port ⓘ

^ Additional TCP ports

Figure 18: Paramètres d'Ingress pour PostgreSQL dans Azure Container App

Puis finalement, il faut déployer l'instance:

Create Container App ...

✓ Passed

Basics Container Ingress Tags Review + create

Project details

Subscription	Azure for Students
Resource group	ResourceMoverRG-west europe-eastus-eus2
Name	postgres

Container Apps Environment

Region	canadaeast
Container Apps Environment	managedEnvironment-dvna-b2d1

Container

Name	postgres
Image source	Public
Registry login server	docker.io
Image and tag	postgres:13
Command	
Arguments	
Workload profile type	Consumption
Number of CPU cores	0.5
Memory size (Gi)	1
Ingress settings	Limited to Container Apps Environment
Ingress type	TCP
Target port	5432
Exposed port	5432

Create < Previous Next Download a template for automation

Figure 19: Résumé avant déploiement d'une instance PostgreSQL dans Azure Container App

Maintenant que PostgreSQL est configuré et que l'ingress est défini pour accepter les connexions TCP sur le port 5432, la prochaine étape est de configurer votre application **DVNA** pour se connecter à cette instance PostgreSQL.

2.Étapes pour déployer DVNA dans Azure Container

Le déploiement de l'application **DVNA (Damn Vulnerable Node Application)** dans un **conteneur Azure** nécessite une série d'étapes pour configurer correctement le conteneur et le connecter à une base de données PostgreSQL déjà déployée. Ce processus inclut la définition des ressources du conteneur, la sélection de l'image Docker, et la configuration des variables d'environnement pour établir la connexion avec la base de données. Voici les étapes à suivre pour déployer **DVNA** dans **Azure Container Instances**.

1. **Nommer le conteneur** : Définir le nom du conteneur en tant que "dvna" pour identifier l'application DVNA.
2. **Sélectionner l'image publique** : Choisir l'option "Public" pour indiquer que l'image DVNA est accessible publiquement.
3. **Sélectionner l'image déployée via GitHub Actions** : Dans notre cas c'est dockerchenak/dvna-tp2:latest
4. **Configurer les variables d'environnement de PostgreSQL**: Ajouter les variables POSTGRES_DB=mydatabase, POSTGRES_USER=chenak, et POSTGRES_PASSWORD pour connecter DVNA à PostgreSQL.

Home > Container Apps >

Create Container App

Select a quickstart image for your container, or deselect quickstart image to use an existing container.

Use quickstart image ☐

Container details

Name *

Image source ☐ Azure Container Registry ☒ Docker Hub or other registries

Image type ☒ Public ☐ Private

Registry login server *

Image and tag *

Command override

Arguments override

Development stack-specific features

When you select a specific development stack, you get additional features tailored to that stack—optimizing Container Apps to perform for your unique settings.

Development stack

Container resource allocation

Choose the workload profile for this app. You can adjust the CPU and memory allocation for this app up to the workload profile limit. [Learn more](#)

Workload profile *

CPU and Memory *

Environment variables

Name	Value	Delete
POSTGRES_DB	mydatabase	Delete
POSTGRES_USER	chenak	Delete
POSTGRES_PASSWORD		Delete

Figure 20: Création de l'application DVNA dans Azure Container

Activer Ingress :

1. **Enabled:** pour permettre au conteneur DVNA de recevoir des connexions réseau.
2. **Configurer l'accès au trafic :** Sélectionnez **Accepting traffic from anywhere** pour permettre l'accès à votre conteneur DVNA depuis n'importe quelle source externe, y compris Internet.
3. **Choisir le type d'ingress :** Sélectionnez **HTTP** pour autoriser l'accès via des requêtes web standard.
4. **Configurer le transport :** Laissez l'option **Auto** sélectionnée pour permettre à Azure de choisir automatiquement le transport le plus approprié pour les données.
5. **Gérer les connexions non sécurisées :** Ne cochez pas la case **Insecure connections (Allowed)** si vous souhaitez uniquement autoriser des connexions sécurisées.
6. **Définir le port cible :** Indiquez **3000** dans le champ **Target port** pour définir ce port comme point d'entrée pour l'application DVNA.

[Home](#) > [Container Apps](#) >

Create Container App ...

Basics ● Container Ingress Tags Review + create

Application ingress settings

Enable ingress for applications that need an HTTP or TCP endpoint.

Ingress ⓘ ☒ Enabled

Ingress traffic ☐ Limited to Container Apps Environment
Select this option if you want to restrict traffic to this container app from within the Container App Environment

☒ Accepting traffic from anywhere
Select this option if you want to allow traffic to this container app from anywhere

Ingress type ⓘ ☒ HTTP

☐ TCP

Transport ▼

Insecure connections ☐ Allowed

Target port ⓘ

Session affinity ⓘ ☐ Enabled

^ Additional TCP ports

Figure 21: Configuration de l'ingress pour la Container App DVNA sur Azure

Puis finalement, on crée l'application:

[Home](#) > [Container Apps](#) >

Create Container App ...

✓ Passed

Basics Container Ingress Tags Review + create

Project details

Subscription	Azure for Students
Resource group	dvna
Name	dvnaa

Container Apps Environment

Region	canadaeast
Container Apps Environment	managedEnvironment-dvna-b2d1

Container

Name	dvnaa
Image source	Public
Registry login server	docker.io
Image and tag	dockerchenak/dvna-tp2:latest
Command	
Arguments	
Workload profile type	Consumption
Number of CPU cores	0.5
Memory size (Gi)	1
Ingress settings	Accepting traffic from anywhere
Ingress type	HTTP
Transport	Auto
Insecure connections	Not allowed
Port	3000

[Create](#) [< Previous](#) [Next](#) [Download a template for automation](#)

Figure 22: Résumé de la configuration de la Container App DVNA sur Azure

On peut ensuite accéder au déploiement à partir de l'URL :
<https://dvna.blueisland-aa8261f7.canadaeast.azurecontainerapps.io/login>

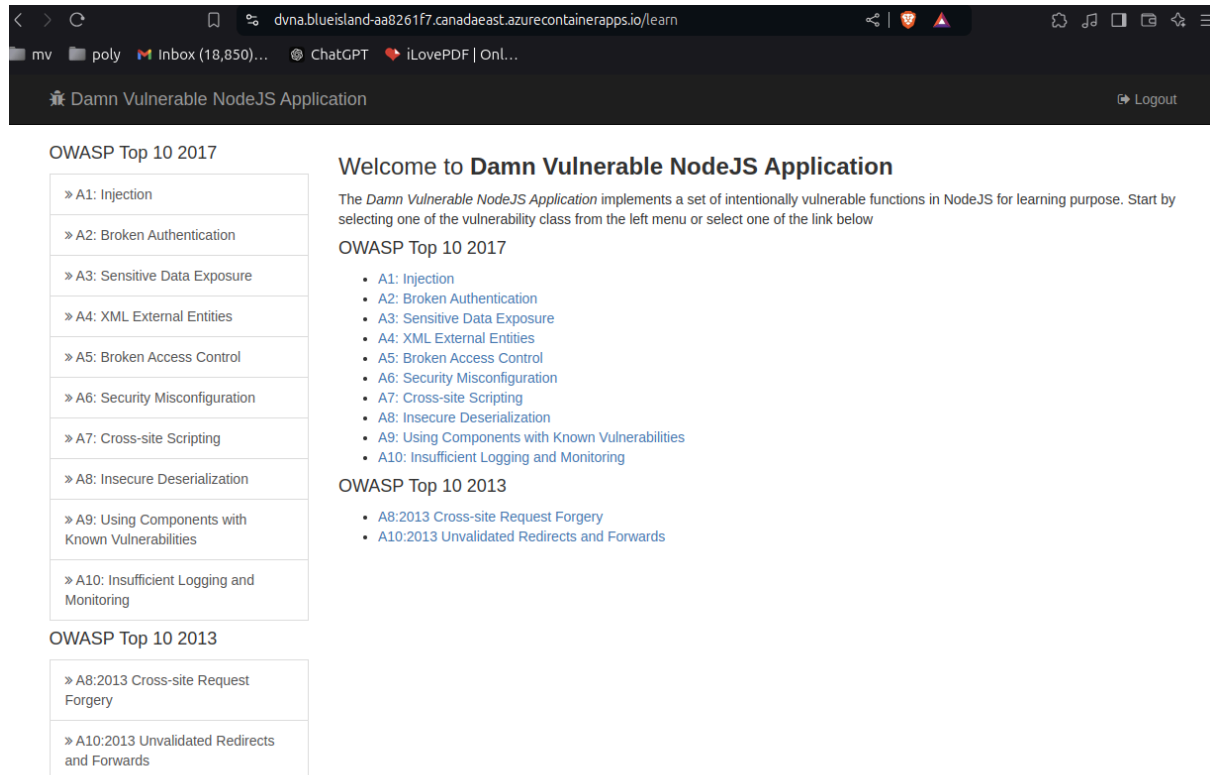


Figure 23: Interface de connexion de l'application DVNA déployée sur Azure

lien rapport : <https://jucetp.github.io/dvna/>

Conclusion

En résumé, l'analyse de sécurité de l'application a révélé plusieurs vulnérabilités critiques, notamment **l'absence de jetons Anti-CSRF**, **des politiques de sécurité laxistes** comme la directive wildcard dans le CSP, et **l'absence de certains en-têtes** de sécurité essentiels. Ces faiblesses exposent l'application à des attaques de type **XSS** et **Clickjacking**, compromettant potentiellement les données des utilisateurs et l'intégrité de leurs sessions. Grâce à l'intégration des outils comme **OWASP ZAP** et **Orchestron** dans le pipeline CI/CD, les vulnérabilités sont désormais détectées automatiquement lors de chaque déploiement, facilitant ainsi une gestion proactive des failles de sécurité. Afin d'améliorer davantage la sécurité de l'application, il est recommandé de **renforcer les en-têtes** de sécurité, **d'implémenter des jetons Anti-CSRF**, et d'appliquer des **Content Security Policies** strictes. De plus, l'utilisation de scanners de sécurité supplémentaires comme **SonarQube** et **Dependabot** permettent de maintenir une surveillance continue des dépendances et de la qualité du code. Enfin, il serait bénéfique d'envisager des mécanismes de **surveillance post-déploiement** sur Azure pour détecter et répondre rapidement à toute tentative d'intrusion en temps réel.